

Normalização

Prof. Cloves Rocha

Ciência da Computação - CC

Roteiro

- Anomalias (ou Redundâncias)
- Normalização (Formas Normais)
- Exercícios

Anomalias

- Anomalias são mudanças em dados que podem gerar uma inconsistência no banco de dados relacional.
- Uma inconsistência é geralmente representada por situações em que dados que deveriam ser iguais, apresentam valores diferentes em várias tabelas do banco de dados.
- Por exemplo, o valor de compra de um produto deve ser o mesmo valor armazenado nas tabelas recibo e nota fiscal.
- Desse modo, se os valores forem diferentes, entende-se geralmente que existe uma inconsistência.
- Tal inconsistência geralmente aparece quando o banco de dados é projetado de forma inadequada.

Anomalias

AgenciaFuncionario	
FuncN	INT
Nome	VARCHAR(100)
Cargo	VARCHAR(45)
Salario	DECIMAL(2)
NumAg	INT
Endereco	VARCHAR(250)
Telefone	VARCHAR(250)
Indexes	

FuncN	Nome	Cargo	Salário	NumAg	Endereço	Tel
25	Luiz	Caixa	2000	1632	Prudente de Moraes, 15	3605-5223
30	Ricardo	Gerente	5000	1668	Hermes da Fonseca, 20	3605-5223
31	Josita	Caixa	2000	1632	Prudente de Moraes, 15	4225-5889
32	Francisca	Gerente	5600	1632	Prudente de Moraes, 15	4225-5889
33	Andréia	Caixa	2300	1668	Hermes da Fonseca, 20	5223-8556

Anomalias

- Assim, na imagem vista anteriormente, temos uma tabela chamada AgenciaFuncionario, que armazena os dados dos funcionários e de agências bancárias.
- Como essas duas informações (funcionário e agência) são armazenadas na mesma tabela, é possível também saber onde cada funcionário trabalha (agência, endereço, telefone).
- Ou seja, à primeira vista, a tabela AgenciaFuncionario parece ser uma ótima opção para reduzir o número de tabelas e aumentar a velocidade do sistema, entretanto, tal solução pode gerar várias anomalias.

Anomalias de Inserção

- Uma **anomalia de inserção** acontece quando, ao inserir um dado, este pode gerar uma inconsistência no banco de dados.
- No exemplo da imagem, para inserir os detalhes (NumAg, Endereço, Tel.) de um novo funcionário, você deve tomar cuidado para usar exatamente os dados já cadastrados por outros funcionários.
- Por exemplo, um novo funcionário para a agência 1668 deve usar exatamente o mesmo endereço dos outros dois funcionários que também trabalham nessa agência.
- Se isso não for feito, teremos um problema de inconsistência de dados, no qual dois funcionários que trabalham na mesma agência possuem endereços diferentes.

Anomalias de Remoção

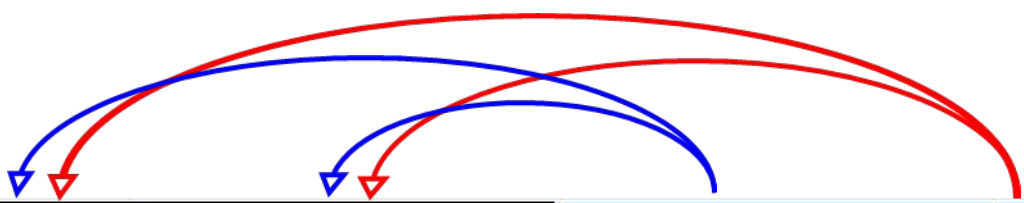
- Uma **anomalia de remoção** acontece quando, ao remover um registro, você pode gerar inconsistência no banco de dados.
- No exemplo da imagem, uma anomalia de remoção acontece quando, por exemplo, removemos o funcionário de número 30.
- Nesse caso, o objetivo é apenas remover os dados do funcionário e preservar os dados da agência 1668. Entretanto, da forma como a tabela está estruturada, os dados da agência também são removidos.

Anomalias de Atualização

- Uma **anomalia de atualização** acontece quando, ao atualizar um registro, você pode gerar inconsistência no banco de dados.
- No exemplo da imagem, uma anomalia de atualização acontece quando, por exemplo, modificamos o endereço da agência 1632.
- Nesse caso, teremos que atualizar o endereço de todos os funcionários da agência 1632. Caso contrário, teremos uma inconsistência no banco de dados no qual funcionários da mesma agência possuem endereços diferentes.
- Note que todas as anomalias acontecem devido à existência de redundância de informação na tabela AgenciaFuncionario. Por exemplo, o mesmo endereço de uma agência é armazenado várias vezes quando ele poderia ser armazenado apenas uma vez.
- Assim, para evitar as anomalias é preciso evitar a redundância. A redundância é evitada através da **normalização das tabelas**.

Conceitos de Dependência Funcional

- A **dependência total ou completa** ocorre quando um ou mais atributos (colunas) de uma relação (tabela) dependem totalmente da chave primária (composta).

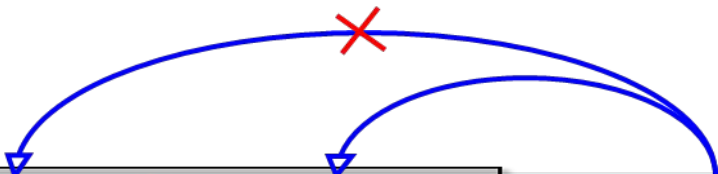


<u>Nt_Fiscal</u>	<u>Cd_Produto</u>	Qt_Vendida	Pr_Venda
1001	101	10	100.00
1001	102	15	150.00
1001	103	20	200.00

The diagram illustrates functional dependencies using red and blue arrows. Red arrows originate from the primary key columns (Nt_Fiscal and Cd_Produto) and point to the non-key attributes (Qt_Vendida and Pr_Venda), indicating total functional dependencies. Blue arrows originate from the primary key columns and point to the primary key columns themselves, indicating self-dependencies.

Conceitos de Dependência Funcional

- Dizemos que existe **dependência parcial** quando atributos (colunas) de uma relação (tabela) não dependem da totalidade da chave primária (composta).



<u>Nt_Fiscal</u>	<u>Cd_Produto</u>	Nm_Produto	Qt_Vendida	Pr_Venda
1001	101	Cabo Vídeo USB	10	100.00
1001	102	Mouse Ótico	15	150.00
1001	103	HD 500Gb	20	200.00

Conceitos de Dependência Funcional

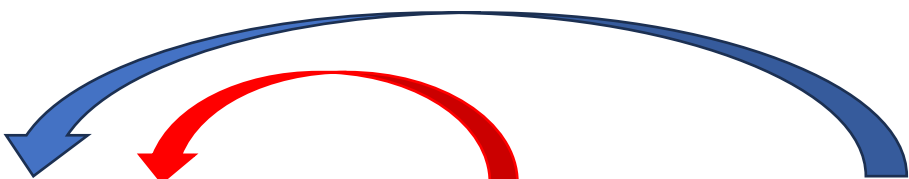
- A **dependência funcional transitiva** ocorre quando um atributo (coluna) não-chave não depende diretamente da chave primária da tabela (nem mesmo parcialmente), mas depende de um outro atributo (coluna) não-chave na tabela.



<u>Num_Pedido</u>	Prazo_Entrega	<i>Cod_Vendedor (FK)</i>	Nome_Vendedor
100120	15	50100001	João Ricardo
100121	30	50100002	José Antônio
100122	5	50100003	Carlos

Conceitos de Dependência Funcional

- A **dependência funcional multivalorada** ocorre quando, para cada valor de um atributo, existe um conjunto de valores para outros atributos que estão associados a ele, mas que são independentes entre si.



<u>Modelo</u>	Ano	Cor
Gol	2016	Prata
Uno	2016, 2015	Preto, Prata
Fox	2016, 2014	Vermelho, Branco

Normalização

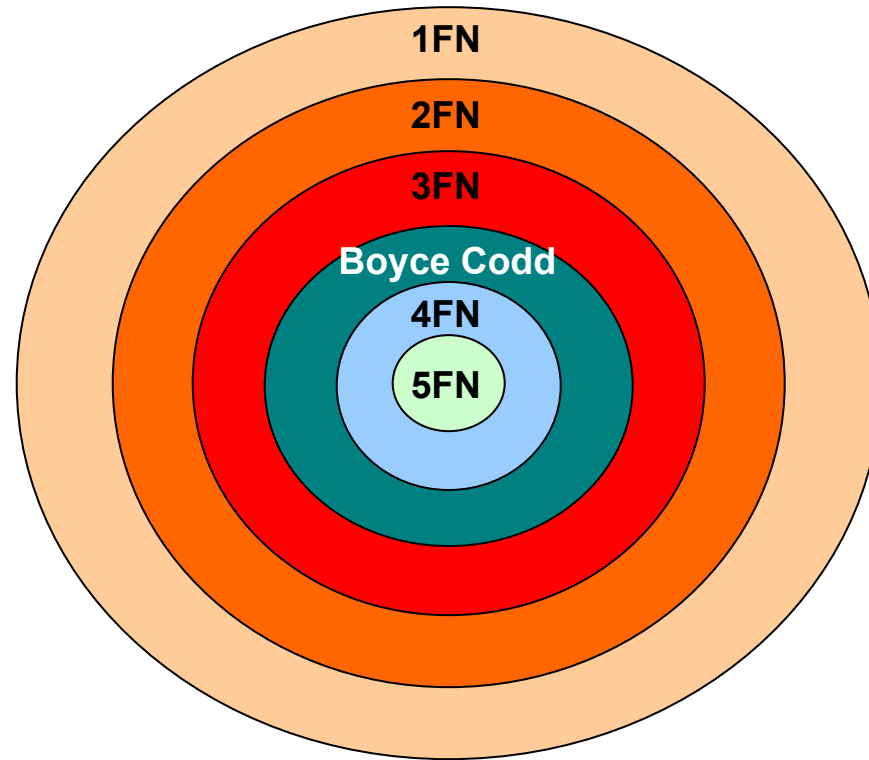
- É a técnica que tem por objetivo principal a eliminação ou minimização, de redundâncias (ou anomalias).
- A redundância de dados, tende a gerar os seguintes problemas:
 - Inconsistência;
 - Queda de performance;
 - Aumento de custos.

Normalização

- **Objetivos:**

- Minimizar redundâncias
- Eliminar anomalias de atualização
- Prover o melhor caminho de acesso
- Assegurar consistência na manutenção de modelo de dados
- Evitar dados não identificáveis através de definição rigorosa de identificadores e relacionamentos

Normalização



Normalização

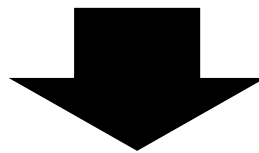
PROJETO (CodProj, Tipo, Descricao, (CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado))

Exemplo tem uma tabela embutida (atributo composto)

CodProj	Tipo	Descricao	Empregado					
			CodEmpregado	Nome	Categoria	Salario	DataInicio	TempoAlocado
PJ554	Desenvolvimento	Sistema de estoque	1	João	A1	6	01/01/2019	12
			2	Maria	A1	6	01/01/2019	12
			7	José	B2	4	01/01/2019	12
			8	Angelo	B3	2	30/03/2019	9
			9	Márcia	A2	5	15/04/2019	6
			10	Pedro	A2	5	15/04/2019	6
PJ529	Desenvolvimento	Sistema de compra	1	João	A1	6	01/01/2020	8
			2	Maria	A1	6	01/01/2020	3
			7	José	B2	4	01/01/2020	6
			8	Angelo	B3	2	01/01/2020	5
			9	Márcia	A2	5	15/11/2019	5
			10	Pedro	A2	5	15/11/2019	8
PF38	Correção	Problema no upload	3	Augusto	A1	6	01/01/2018	3
			4	Joana	B2	4	01/01/2018	1
			5	Alex	B3	2	01/01/2018	5
			6	Sophie	B3	2	01/01/2018	2
			7	José	A2	4	01/01/2020	6
			8	Angelo	B3	2	01/01/2020	5

Normalização - 1FN (Primeira Forma Normal)

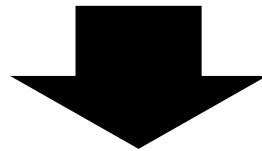
- Os atributos que compõem uma tabela devem ser todos atômicos (não podem ser divisíveis).
- Não devem conter outras tabelas embutidas.
- **PROJETO** (CodProj, Tipo, Descricao, (CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado))



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado).

Normalização - 2FN (Segunda Forma Normal)

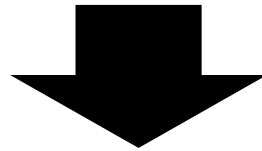
- Deve estar na 1FN e não possuir **dependências parciais**.
- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado).



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Salario).

Normalização - 3FN (Terceira Forma Normal)

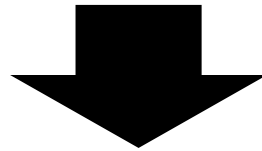
- Deve estar na 2FN e não possuir **dependências transitivas**.
- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Salario).



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **CATEGORIA** (Categoria, Salario).

Normalização - 4FN (Quarta Forma Normal)

- Deve estar na 3FN e não possuir **dependências multivaloradas**.
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Fones(1,n));



- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **FONES** (CodEmpregado, NFone).

Normalização - Resumo

- Eliminar atributos não atômicos □ 1FN
- 1FN + Eliminar dependências funcionais parciais □ 2FN
- 2FN + Eliminar dependências transitivas □ 3FN
- 3FN + Eliminar dependências multivaloradas □ 4FN

Exercícios 1

Apresente as formas normais até a 4FN, quando for o caso, para as tabelas:

a) Vendedor (nroVend, sexoVend, nomeVend, {Cliente (nroCli, endCli, nomeCli)})

- Observações adicionais:
 - Um vendedor pode atender diversos clientes, e um cliente pode ser atendido por diversos vendedores.

Exercícios 2

Apresente as formas normais até a 4FN, quando for o caso, para as tabelas:

b) Aluno(nroAluno, nomeDepto, siglaDepto, codDepto, codOrient, nomeOrient, foneOrient, codCurso)

- Observações adicionais:
 - Um aluno somente pode estar associado a um departamento.
 - Um aluno cursa apenas um único curso.
 - Um aluno somente pode ser orientado por um único orientador

Exercícios 3

Apresente as formas normais até a 4FN, quando for o caso, para as tabelas:

c) OrdemCompra(codOrdemCompra, dtEmissao, codFornecedor, nomeFornecedor, enderecoFornecedor, {ItemOrdem(codItem, {Material(codMaterial, descricaoMaterial, qtComprada, valorUnitario))}, valorTotalItem)}, valorTotalOrdem).

- Observações adicionais:
 - Uma ordem de compra possui apenas um fornecedor.
 - Um item de ordem possui vários materiais.

Referências

- COUGO, Paulo. Modelagem conceitual e projeto de banco de dados. São Paulo: Elsevier, 2017.
- MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. Projeto de banco de dados: uma visão prática. 17. ed. São Paulo, SP: Érica, 2013.
- RAMAKRISHNAN, Raghu; GEHRKE, Johannes. Sistemas de gerenciamento de banco de dados. São Paulo, SP: McGraw-Hill, c2008.

COMPLEMENTAR

- DATE, C. J. Introdução a sistemas de bancos de dados. [8. ed.], 7. tiragem. Rio de Janeiro, RJ: Elsevier, c2004.
- DATE, C. J. Projeto de banco de dados e teoria relacional. São Paulo: Novatec, 2015.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. 3. reimp. São Paulo, SP: Pearson Addison Wesley, 2014.
- OPPEL, Andy; SHELDON, Robert. SQL: um guia para iniciantes. 3. ed. Rio de Janeiro, RJ: Ciência Moderna, 2009.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 6 ed. São Paulo, SP: Makron books, 2012.

Normalização

Prof. Cloves Rocha

Ciência da Computação - CC